Scalable Multi-Robot Simulation in ROS 2 and Gazebo

Contributor: Jiayi Cai (Zhejiang University)
Mentor: Addisu Taddese
Google Summer of Code 2026

CONTENTS

01 | Background & Motivation

02 | Model Namespace

03 | Automated Bridge

04 | Multi-Robot TF

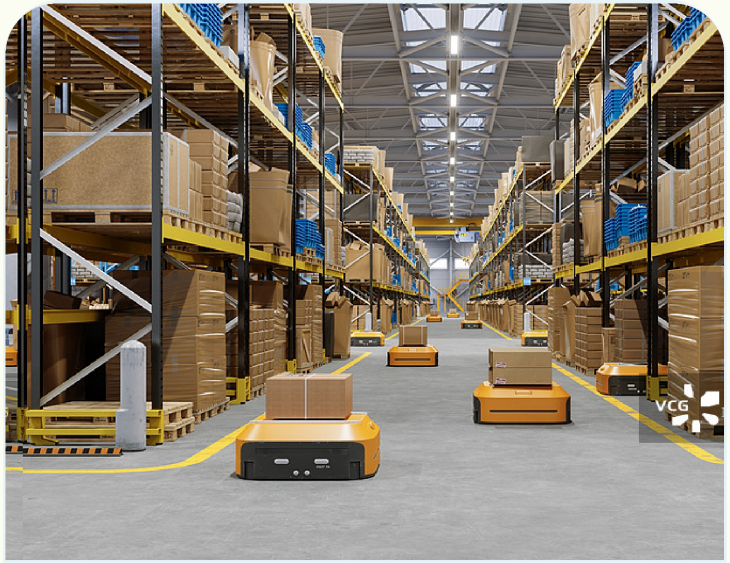
05 | Warehouse Demo

01

Background & Motivation

Multi-Robot Simulation: Where Is It Used?

Multi-robot simulation enables the development and testing of coordinated robotic systems across a wide range of real-world applications.



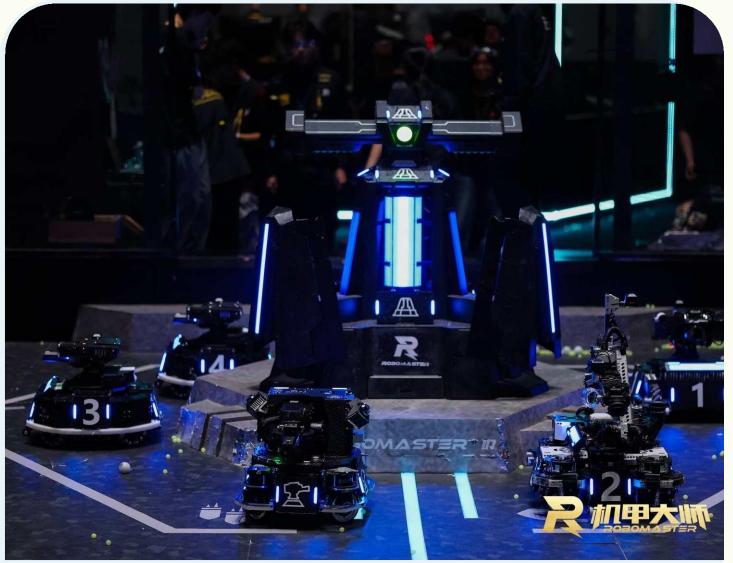
Warehouse Automation

Coordinated transportation and logistics



Multi-Robot Swarms

Collaborative exploration, navigation and coordination

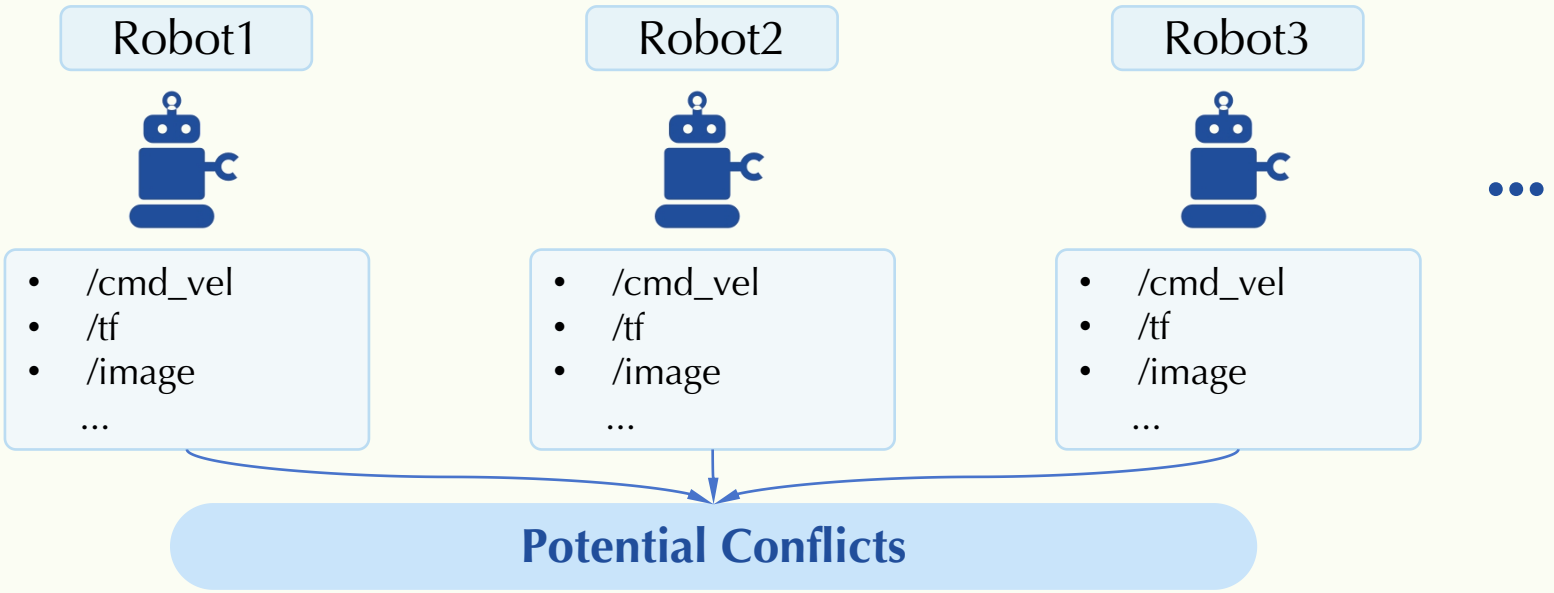


Robotics Competitions

Multi-Robot coordination and competition such as RoboMaster

Challenge 1: Communication Isolation

How can we **reuse the same robot model** while keeping its **topics and services independent**?



Each robot needs **independent communication interfaces**

Control
e.g., /cmd_vel, /enable

Feedback
e.g., /odom, /tf

Sensor Data
e.g., /points, /image

Existing Approaches

1 Separate Robot Models

Create individual model definitions with different topic and service names.

robot1.sdf
robot2.sdf
robot3.sdf
...

```
<model name="robot1">
  ...
  <topic>robot1/cmd_vel</topic>
  ...
</model>
```

Significant code duplication and maintenance overhead.

2 Xacro Templates

Use Xacro to generate robot models with configurable topic and service names.

```
<xacro:macro name="robot1" params="ns">
  ...
  <topic>${ns}/cmd_vel</topic>
  ...
</xacro:macro>
```

Requires additional knowledge of Xacro and template configuration

Challenge 2: Repetitive ROS-Gazebo Bridge Configuration

Manual Bridge Configuration (YAML)

```
# Robot1
- topic_name: "/robot1/cmd_vel"
  ros_type_name: "geometry_msgs/msg/Twist"
  gz_type_name: "gz.msgs.Twist"
  lazy: true
  direction: ROS_TO_GZ
  ...

# Robot2
- topic_name: "/robot2/cmd_vel"
  ros_type_name: "geometry_msgs/msg/Twist"
  gz_type_name: "gz.msgs.Twist"
  lazy: true
  direction: ROS_TO_GZ
  ...

# Robot3
- topic_name: "/robot3/cmd_vel"
  ...
```

Bridges For Robot 1

Bridges For Robot 2

Bridges For Robot 3

- As the number of robots grows, manually configuring ROS-Gazebo bridges becomes **increasingly repetitive** and **difficult to maintain**.

Two Challenges, Two Solutions

1 Challenge 1: Communication Isolation

Reusing robot models while keeping communication interfaces **independent**.

Model Namespace Support

Organize topics and services under model namespaces.

Robot1
namespace: {name}_ns

- **/robot1_ns**/cmd_vel
- **/robot1_ns**/tf
- **/robot1_ns**/image
- **/robot1_ns**/points

...

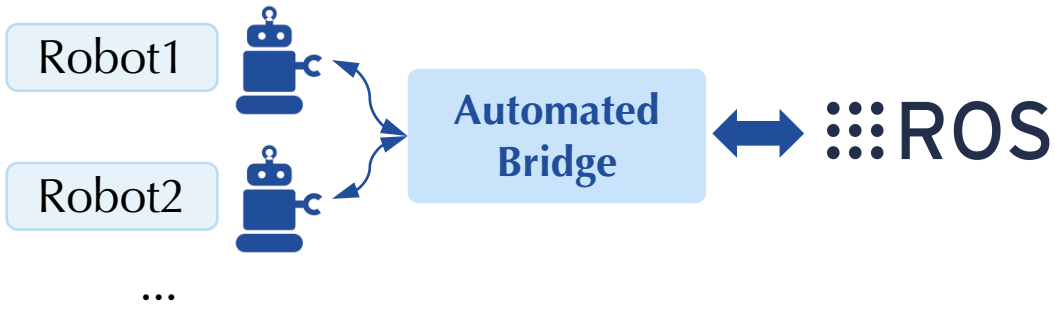
Reuse robot models without manually renaming every topic and service.

2 Challenge 2: Repetitive Bridge Configuration

Manually configuring **similar** ROS-Gazebo bridges for every robot.

Automated ROS-Gazebo Bridge

Automatically discover and bridge supported ROS 2 and Gazebo interfaces.



Reduce repetitive bridge configuration and support dynamically spawned robots.

Model Namespace: Beyond Multi-Robot Simulation

Addressing inconsistent plugin naming conventions while preserving flexibility and compatibility

1 Inconsistent Naming Rules Across Gazebo Plugins

Plugin	Configuration	Resolved Name	Default Name
DiffDrive	<topic>	<topic>	/model/{model_name}/cmd_vel
JointController	<topic>	<topic>	/model/{model_name}/joint/{joint_name}/cmd_vel
	<sub_topic>	/model/{model_name}/<sub_topic>	
Thruster	<namespace>	<namespace>/<topic>	/model/<namespace>/joint/{joint_name}/cmd_vel
	<topic>		
Imu	<topic>	<topic>	{scoped_name}/imu

Example defualt name: /world/multi_robot/model/vehicle_group_1/model/vehicle_1/link/chassis/sensor/imu/imu

Inconsistent Rules → Learing Cost

Log Default Name → Inconvenient

2 Why Introduce Namespace Instead of Using Model Names?

Flexibility

Model names are **required** for every model entity, while namespaces are **optional** and user-defined, allowing users to organize communication interfaces more **freely**.

Backward Compatibility

Automatically using model names as prefixes would change existing default and custom naming behavior, which could bring inconvenience to current users.

02

Model Namespace

Model Namespace: Namespace Resolution Rules

Namespaces are resolved from the current model up through the model hierarchy.

```
<world name="default">
  <model name="robot_group1" namespace="{name}_ns">
    <!-- Final Namespace: "robot_group1_ns" -->
    <model name="robot1"/>

    <!-- Final Namespace: "robot_group1_ns/robot2_ns" -->
    <model name="robot2"
           namespace="{name}_ns"/>

    <!-- Final Namespace: "/robot3_ns" -->
    <model name="robot3"
           namespace="/{name}_ns"/>
  </model>

  <model name="robot_group2">
    <!-- Final Namespace: "" -->
    <model name="robot1"/>
  </model>
</world>
```

1. **Start from the current model**

   Use the model's own namespace if non-empty. The `{name}` placeholder is replaced with the model's final name.

2. **Move to the parent model**

   Continue searching up the entity tree hierarchy.

3. **Stop at an absolute namespace**

   If a namespace starts with `"/"` stop searching upward.

4. **Combine namespace segments**

   Join the collected namespaces using `"/"`.

Model Namespace: Topic / Service Name Resolution Rules

Namespace-aware topic resolution while preserving existing behavior.

1 Without Namespace (existing behavior)

```
<model name="robot1">
  <plugin filename=gz-sim-diff-drive-system name=...>
    <!-- Option 1: no <topic> (use defualt) -->
    <!-- Option 2: custom topic -->
    <topic>custom_cmd_vel</topic>
    <!-- or -->
    <topic>/custom_cmd_vel</topic>
  </plugin>
</model>
```

2 With Namespace (new feature)

```
<model name="robot1" namespace="{name}_ns">
  <plugin filename=gz-sim-diff-drive-system  name=...>
    <!-- Option 1: no <topic> (use defualt) -->
    <!-- Option 2: custom topic -->
    <topic>custom_cmd_vel</topic>
    <!-- or -->
    <topic>/custom_cmd_vel</topic>
  </plugin>
</model>
```

1 Without Namespace (Backward Compatible)

Configuration	<...> in SDF	Resolved Name
Default Name		{default_prefix}/{default_suffix}
Custom Name (**relative**)	custom_topic	{custom_topic}
Custom Name (**absolute**)	/custom_topic	{custom_topic}

2 With Namespace

namespace="robot1_ns"

Configuration	<...> in SDF	Resolved Name
Default Name		{namespace}/{default_suffix}
Custom Name (**relative**)	custom_topic	{namespace}/{custom_topic}
Custom Name (**absolute**)	/custom_topic	{custom_topic}

Model Namespace: Topic / Service Name Resolution Rules

Namespace-aware topic resolution while preserving existing behavior.

1 Without Namespace (existing behavior)

```
<model name="robot1">
  <plugin filename=gz-sim-diff-drive-system name=...>
    <!-- Option 1: no <topic> (use defualt) -->
    <!-- Option 2: custom topic -->
    <topic>custom_cmd_vel</topic>
    <!-- or -->
    <topic>/custom_cmd_vel</topic>
  </plugin>
</model>
```

2 With Namespace (new feature)

```
<model name="robot1" namespace="{name}_ns">
  <plugin filename=gz-sim-diff-drive-system  name=...>
    <!-- Option 1: no <topic> (use defualt) -->
    <!-- Option 2: custom topic -->
    <topic>custom_cmd_vel</topic>
    <!-- or -->
    <topic>/custom_cmd_vel</topic>
  </plugin>
</model>
```

1 Without Namespace (Backward Compatible)

Configuration	<...> in SDF	Resolved Name
Default Name		**/model/robot1**/cmd_vel
Custom Name (**relative**)	custom_cmd_vel	custom_cmd_vel
Custom Name (**absolute**)	/custom_cmd_vel	/custom_cmd_vel

2 With Namespace

namespace="robot1_ns"

Configuration	<...> in SDF	Resolved Name
Default Name		**robot1_ns**/cmd_vel
Custom Name (**relative**)	custom_cmd_vel	**robot1_ns**/custom_cmd_vel
Custom Name (**absolute**)	/custom_cmd_vel	/custom_cmd_vel

How to Set a Model Namespace

A model namespace can be specified in SDF or when spawning a model at runtime.

1 In SDF Files

Set Namespace as a <model> Attribute

```
<model name="vehicle"
       namespace="{name}_ns">
  ...
</model>
```

Set Namespace as a <include> Element

```
<include>
  <uri>...</uri>
  <name>vehicle</name>
  <namespace>{name}_ns</namespace>
</include>
```

2 Using ros_gz_sim create

Spawn a model with the ros_gz_sim create node and set the namespace with `-entity_namespace`.

```
ros2 run ros_gz_sim create \
  -world multi_robot \
  -file ... \
  -name vehicle \
  -entity_namespace {name}_ns
```

If `-entity_namespace` is not provided or is empty, the namespace behavior follows the source SDF file. Use `-entity_namespace '/'` to ignore any namespace in the SDF file.

3 Using gz_spawn_model

Spawn a model with the gz_spawn_model launch action or launch file and set the namespace with `entity_namespace`.

```
<gz_spawn_model
  world="default"
  entity_name="vehicle"
  entity_namespace="{name}_ns"
  ...>
</gz_spawn_model>
```

If `entity_namespace` is not provided or is empty, the namespace behavior follows the source SDF file. Use `entity_namespace="/"` to ignore any namespace in the SDF file.

4 Using ROS 2 or Gazebo Services

Spawn a model through simulation services such as /gzserver/spawn_entity(ROS 2) or /world/<world>/create (Gazebo), and set the namespace.

```
ros2 service call /gzserver/spawn_entity
simulation_interfaces/srv/SpawnEntity "{
  name: 'vehicle',
  entity_resource: ...,
  entity_namespace: '{name}_ns'
  }}"
```

```
gz service -s /world/multi_robot/create/blocking \
  --reqtype gz.msgs.EntityFactory \
  --reptype gz.msgs.Boolean
  --timeout 5000 \
  --req 'sdf_filename: ...,
         name: "vehicle",
         entity_namespace: "{name}_ns"}'
```

If `entity_namespace` is not provided or is empty, the namespace behavior follows the source SDF file. Use `entity_namespace="/"` to ignore any namespace in the SDF file.

Model Namespace: Current Status & Future Plans

1 Plugin Support

Namespace support has been added to several plugins, and will be extended to all Gazebo Sim plugins.

DiffDrive
Namespace support and unit tests
Merged

MecanumDrive
Namespace support and unit tests
PR Open

All Sensors
Namespace support and unit tests for rendering sensors and sensor plugins
PR Open

Future Plan

Extend namespace support to **all Gazebo plugins**.

2 Runtime Model Creation

Models can be created at runtime with or without a namespace. We will unify the interface to support namespaces in the original creation service.

Current (two creation interface)

Will be backported to older version

gz::msgs::EntityFactoryWithNs	gz::msgs::EntityFactory
/world/{world_name}/create_with_ns	/world/{world_name}/create
Support **entity_namespace**	No **entity_namespace**

After PR #4057 (unified interface)

Will **not** be backported to older versions

gz::msgs::EntityFactoryWithNs	gz::msgs::EntityFactory
/world/{world_name}/create_with_ns	/world/{world_name}/create
Deprecated	Support **entity_namespace**

03

Automated Bridge

Automated Bridge: Configuration and Performance

Minimal configuration, continuous discovery, and low runtime overhead.

1 Configuration

Enable automated bridging and exclude specific interfaces.

```
#example
ros_gz_bridge:
  ros__parameters:
    automated_bridge:
      enable: true
      include_patterns:
        - "/robot1.*"
      exclude_patterns:
        - ".*camera.*"
```

automated_bridge.enable

Enables periodic discovery and automatic bridging of ROS 2 and Gazebo interfaces.

automated_bridge.include_patterns

A list of **regex patterns** to specify (if set) which topics and services are considered for automated bridging.

automated_bridge.exclude_patterns

A list of **regex patterns** to exclude specific topics and services from automated bridging.

2 Key Characteristics

Automatically discover and bridge interfaces at runtime

Continuous Discovery

Supports any startup order, periodically discovers and attempts to bridge newly available topics / services at runtime.

Manual Configuration First

Manually configured bridges **take priority**, while automated bridging handles the **remaining** interfaces.

3 Performance Overhead

Low additional cost even for large multi-robot setups.

CPU Usage(average)



Memory Usage(average)



Test Environment

- Intel Core Ultra 5 125H (14 cores /18 threads)
- 30 GiB RAM
- Ubuntu 24.04.4 LTS
- Kernel 6.8.0-138-generic

- 124 Gazebo topics, 94 Gazebo services
- 115 ROS topics, 44 ROS services
- Average of 60 measurements

Automated Bridge: Discovery and Creation Rules

Automatically discover compatible ROS 2 and Gazebo interfaces and create bridges with the appropriate direction.

1 Discover

Discover available topics and services in ROS 2 and Gazebo.

ROS
- Topics
- Services

GAZEBO
- Topics
- Services

2 Match

Find ROS 2 and Gazebo interfaces that meet the following conditions:

ROS 2	Gazebo
/robot/odom nav_msgs/msg /Odometry	**/robot/odom** gz.msgs.Odometry

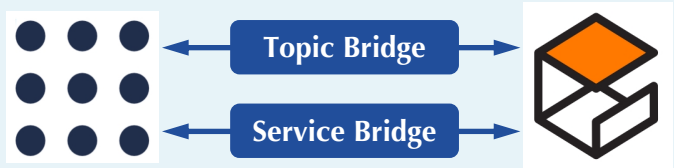
1. Same name
The topic / service name is identical.

2. Unambiguous Types
Each side has exactly one type.

3. Compatible Mapping
A supported ROS-Gz type mapping exists.

3 Create

Automatically establish the required bridges.



- **Topic Bridging**

The bridge direction is determined by the publishers and subscribers from the Gazebo side.

Gazebo Communication	Bridge Direction
Publisher only	GZ_TO_ROS
Subscriber only	ROS_TO_GZ
Both	BIDIRECTIONAL

- **Service Bridging**

ROS 2 side is a client and Gazebo side is a server, so no direction inference is required.



Automated Bridge: Usage and Best Practices

Enable automated bridging via the command line or launch files, with care when running multiple bridge instances.

1 How to Enable

Command Line

```
ros2 run ros_gz_bridge parameter_bridge \
  --ros-args -p automated_bridge.enable:=true \
  -p "automated_bridge.include_patterns:=['/robot1.*']" \
  -p "automated_bridge.exclude_patterns:=['.*camera.*']"
```

Launch File

```
<launch>
  <ros_gz_bridge bridge_name="auto_bridge">
    <param name="automated_bridge.enable" value="true" />
    <param name="automated_bridge.include_patterns"
           value="['/robot1.*']" />
    <param name="automated_bridge.exclude_patterns"
           value="['.*camera.*']" />
  </ros_gz_bridge>
</launch>
```

2 Important Notes

Be careful when running multiple bridge instances to avoid duplicate bridging and message loops.

- **Multiple Automated Bridges**

  Running multiple bridge instances with automated bridge enabled may cause duplicate bridges and message loops.

- **Separate Manual & Automated Bridges**

  Automated bridge cannot detect manual configurations in other bridge instances, and may still attempt to bridge the same interfaces.

Recommended Practice

- Prefer combining manual and automated bridging **in a single bridge**.
- If using separate instances, exclude manually bridged interfaces using **exclude_patterns.**
- If using multiple automated bridge, use **include_patterns** to limit the scope.

3 Current Status

ros_gz/#923/Support automated bridging PR is under review and awaiting merge.

4 Future Work

Reuse automated bridge's internal discovery and inference to support manual configurations with **missing message types or bridge directions.**

04

Multi-Robot TF

Multi-Robot TF: Frame ID Challenge

Topic namespaces isolate communication, but TF messages may use the same frame IDs across different robots.

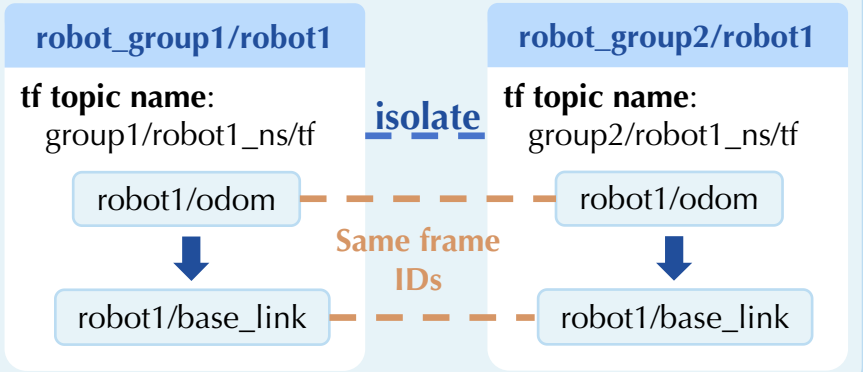
```
<world name="default">
  <model name="robot_group1"
         namespace="group1">
    <include>
      <uri>...<uri>
      <name>robot1<name>
      <namespace> {name}_ns<namespace>
    <include>
  </model>

  <model name="robot_group2"
         namespace="group2">
    <include>
      <uri>...<uri>
      <name>robot1<name>
      <namespace> {name}_ns<namespace>
    <include>
  </model>
</world>
```

1 Default Frame IDs

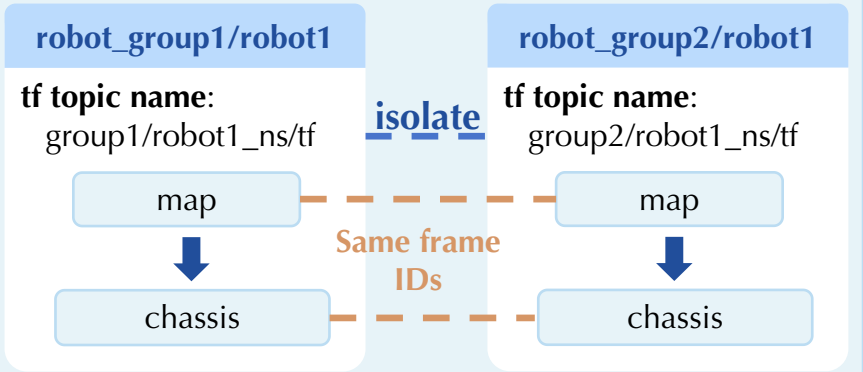
Default frame ID pattern

- frame_id = {model_name}/odom
- child_frame_id = {model_name}/{link_name}



2 Custom Frame IDs

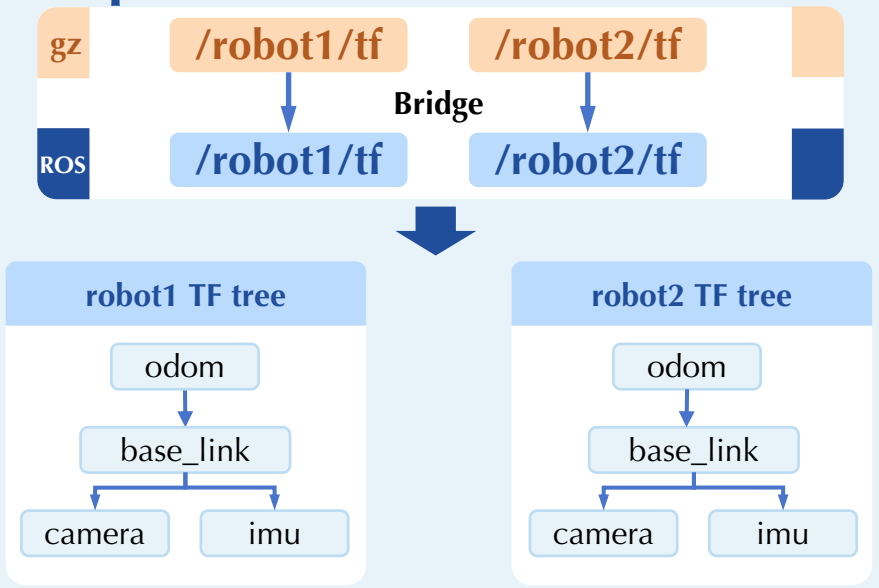
```
<model name="shared_model">
  <plugin>
    <frame_id>map</frame_id>
    <child_frame_id>chassis
    </child_frame_id>
  </plugin>
  ...
</model>
```



Multi-Robot TF: Two Integration Approaches

Choose between independent TF trees and a unified TF tree based on application requirements

1 Independent TF Trees



Implementation

Use separate TF topics and TF tree for each robot.

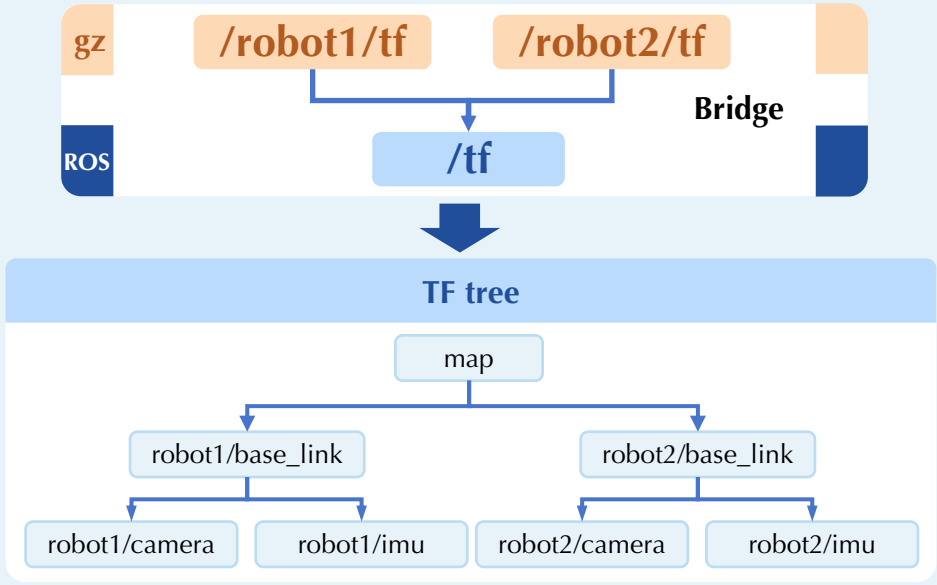
Use case

Independent robot control and state estimation.

Frame IDs can overlap across robots

No conflict as long as each robot uses a separate TF buffer.

2 Unified TF Tree



Implementation

Rewrite TF frame IDs on the ROS 2 side before combining the transforms.

Use case

Multi-robot coordination, global planning and analysis.

Frame IDs must be unique

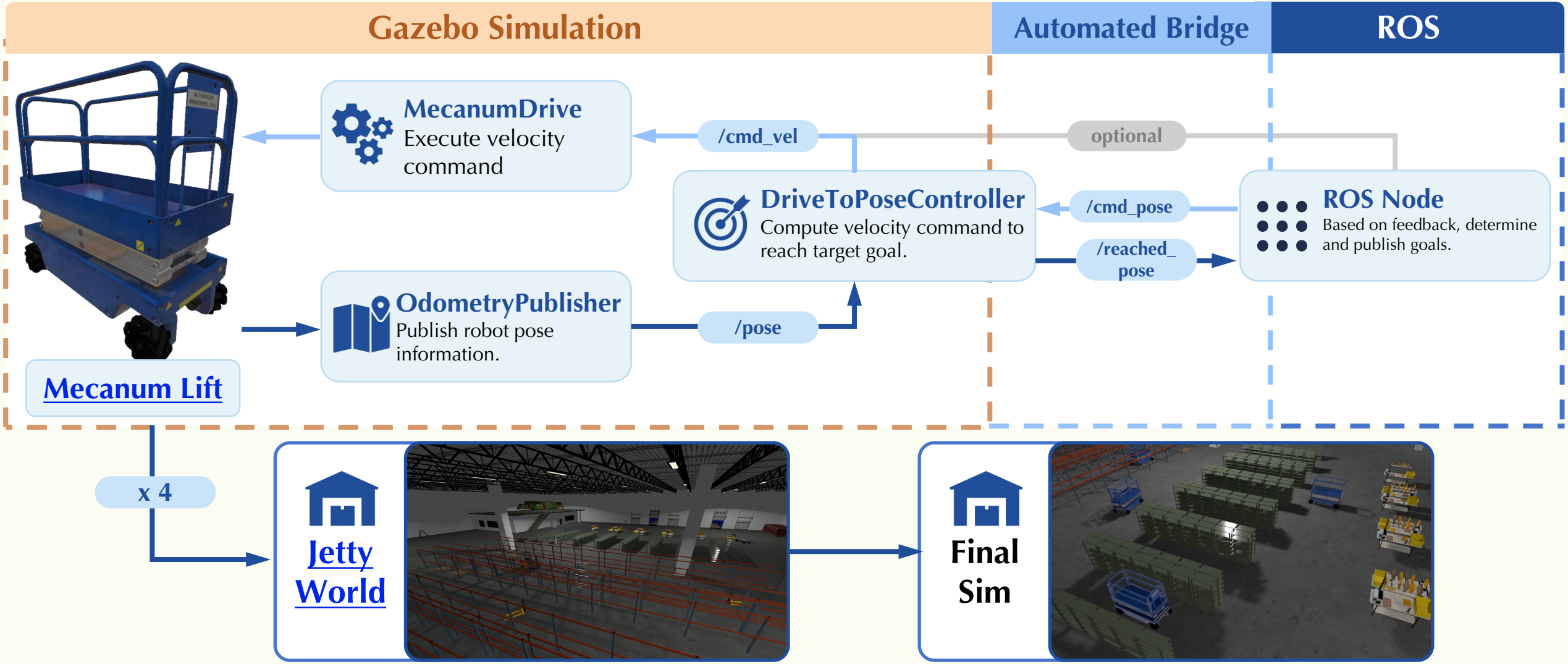
Same frame IDs can cause conflicts when TF is combined.

05

Warehouse Demo

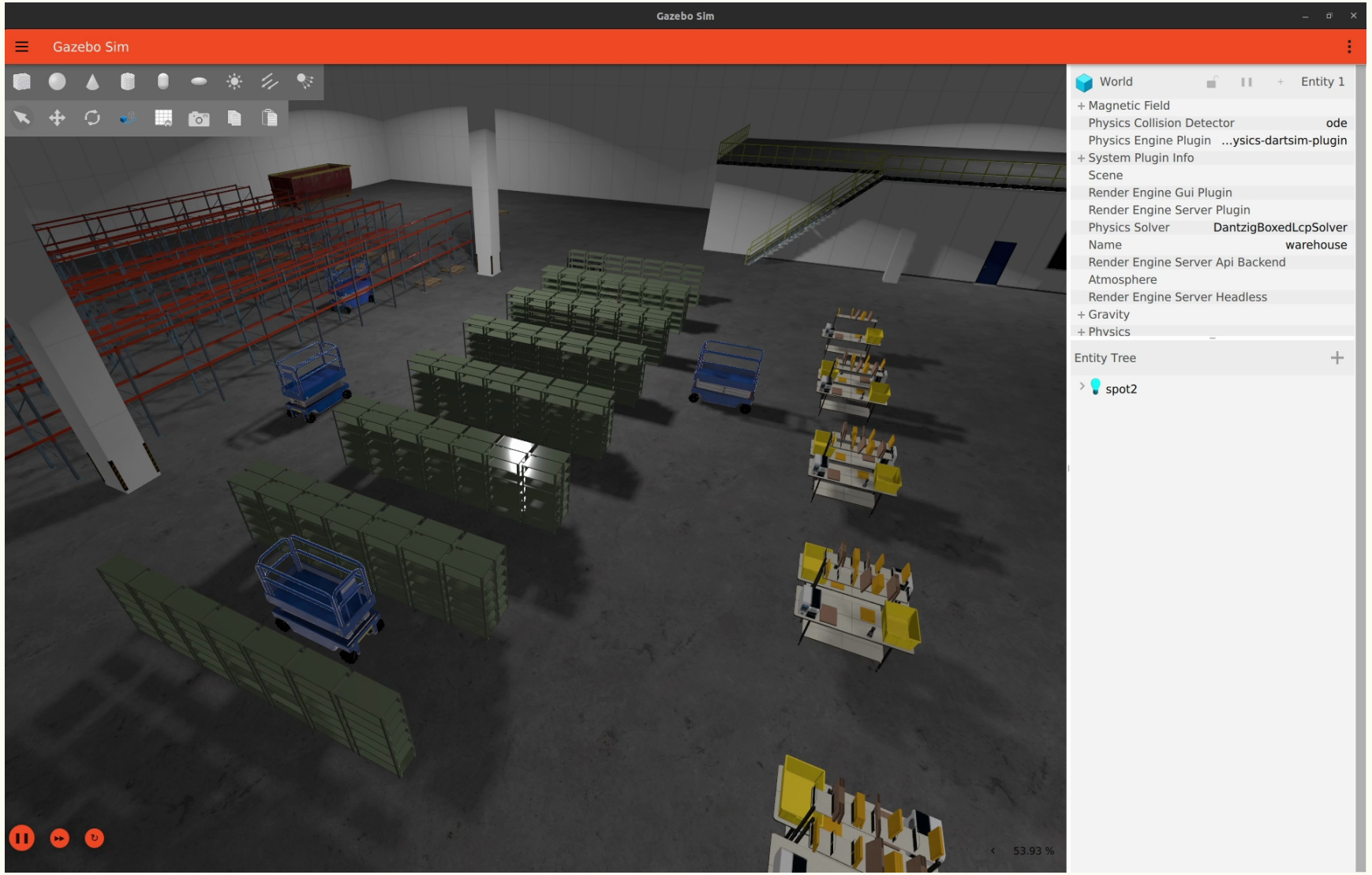
Warehouse Demo: System Overview

Four mecanum-wheel robots sharing one SDF model, with independent ROS 2 control.



Warehouse Demo

Four mecanum-wheel robots sharing one SDF model, with independent ROS 2 control.



Demo Link:
https://github.com/azeey/gsoc2026_multirobot/tree/main/warehouse

Summary & Resources

01 Model Namespace Support

A flexible, namespace-based approach to organizing topic and service names.

Docs

02 Automated ROS-Gz Bridge

Discover and bridge compatible interfaces automatically.

Docs

PR

03 Multi-Robot Intergration

Combine namespace support and automated bridge to simplify multi-robot simulation setup.

Docs

Demo PR

Warhouse Repo

Thanks

Contributor: Jiayi Cai *(jyc12888@gmail.com)*
Mentor: Addisu Taddese
Google Summer of Code 2026